

Day 18 · Data, Bias & Fairness – Differentiated Coding Handout
Advanced STEM · Age 10-11 · Python focus: tuples, Counter, comprehensions, dicts, fairness metrics

Goal Detect class imbalance, repair a biased dataset, and evaluate model fairness with Python.

Key terms Training data = examples a model learns from; bias = systematic unfair skew; class imbalance = one label appears far more often; fairness metric = number for equal treatment.

Warm-up Could true facts still create biased AI? Is 99% cat accuracy good if dog accuracy is 20%? What changes when a city-trained robot moves cities?

1) Core code: count imbalance

Each tuple is an immutable record. Counter tallies labels automatically; _ means the name is intentionally unused.

python

```
from collections import Counter
players = [('Alex','red'), ('Blake','blue'), ('Casey','blue'),
           ('Devon','blue'), ('Emi','red')]
counts = Counter(team for _, team in players)
total = sum(counts.values())
ratio = counts['blue'] / counts['red']
print(f"Ratio blue:red = {ratio:.2f}x")
print(f"Robot prefers: {counts.most_common(1)[0]}")
Trace: Start {} → red:1 → blue:1 → blue:2 → blue:3 → red:2. Dominant class = blue, so a simple
model may over-predict blue.
```

2) Python patterns

python

```
for name, team in players:                # unpack tuple
    print(name, '→', team)
for idx, (name, team) in enumerate(players):
    print(f"#{idx}: {name} is {team}")
reds = [n for n, t in players if t == 'red']
for name, score in zip(['Alex','Blake'], [0.9, 0.7]):
    print(f"{name}: {score:.0%}")
```

Check: What returns from Counter("aabbcc").most_common(1)? Why does Counter avoid a KeyError?

3) Fix biased data

Collect more (best when possible – adds real evidence):

python

```
more = [('Finley','red'), ('Gray','red')]
data = players + more
print(Counter(t for _, t in data))
```

Undersample (randomly drop majority examples; quick but wasteful):

python

```
import random
blues = [p for p in players if p[1] == 'blue']
reds = [p for p in players if p[1] == 'red']
n = min(len(blues), len(reds))
```

```
balanced = random.sample(blues, n) + random.sample(reds, n)
```

Weight classes (keep all data but make rare labels count more):

python

```
counts = Counter(t for _, t in players)
total = sum(counts.values())
weights = {label: total/count for label, count in counts.items()}
print(weights)      # {'blue': 1.67, 'red': 2.5}
```

4) Dict comprehension report

python

```
pct = {label: round(count/total*100, 1)
      for label, count in counts.items()}
for label, percent in pct.items():
    bar = '█' * int(percent // 10)
    print(f"{label:5s} {bar} {percent}%")
```

5) Fairness metrics

Metric	Question it asks
Demographic parity	Do groups receive positive outcomes at equal rates?
Equal opportunity	Among people who should be selected, are groups selected equally?
Calibration	When the model says 70% confident, is it correct 70% of the time for every group?

You cannot always satisfy all fairness definitions at once when base rates differ, so designers must choose and justify a metric.

6) Accuracy by group

python

```
results = [('Alex','red','red'), ('Blake','blue','blue'),
           ('Casey','blue','blue'), ('Devon','blue','red'),
           ('Emi','red','blue'), ('Finley','red','red')]
correct = {t: sum(1 for _,team,pred in results
                 if team == t and pred == t)
          for t in ['red','blue']}
total = Counter(t for _, t, _ in results)
for t in total:
    acc = correct[t] / total[t] * 100
    flag = 'BIAS' if acc < 70 else ''
    print(f"{t}: {correct[t]}/{total[t]} = {acc:.1f}%{flag}")
```

7) Differentiated build

Level	Task	Evidence
Support	Create 12 tuples imbalanced by grade; use Counter to prove ratio.	Counts + ratio
Core	Add percent bars and choose one repair: collect, undersample, or weight.	Explain trade-off
Stretch	Write fairness_report(results, group) for precision and recall.	Test 2 groups

Exit ticket: Name the 3 metrics; explain Amazon's 96% male training set using "pattern" and "generalise"; revisit whether true facts can still create bias.

Homework: Complete one build level + a 150-word response: *"An AI tutoring tool rarely recommends one neighbourhood. Who should fix it, how, and which fairness metric would you measure?"*

Handout: <https://github.com/hue881/AI-Coding-Day18>

Lesson: <https://hue881.github.io>

Activity: <https://hue881.github.io/activity>